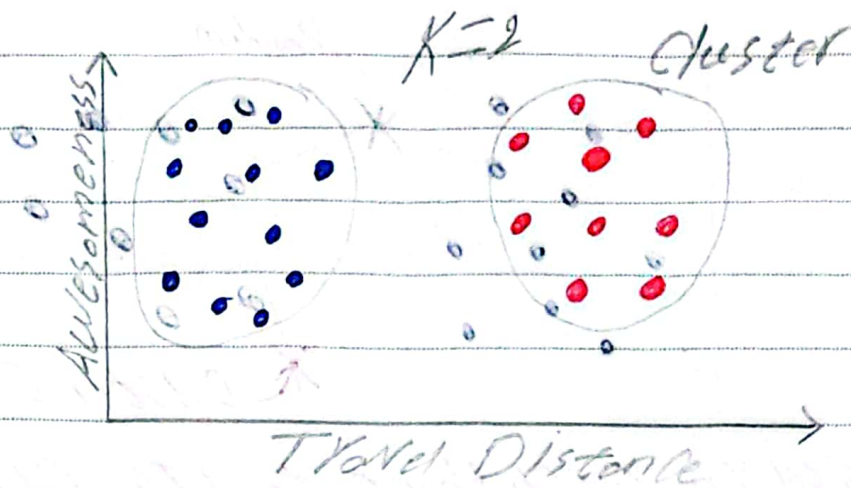
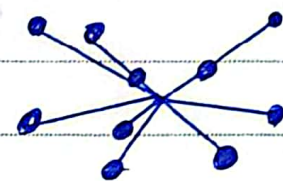


UNSUPERVISED ① Clustering:



→ Number of K
Elbow Method:

$K=1$ avg. distance = 1.261

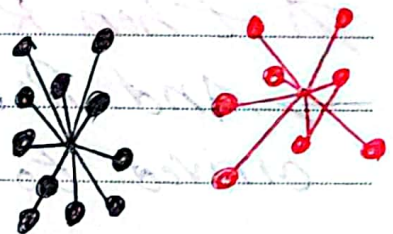


$K=2$ avg dist = .923

$K=3$ avg dist = .639

$K=4$ avg dist = .382

$K=5$ avg dist = .348

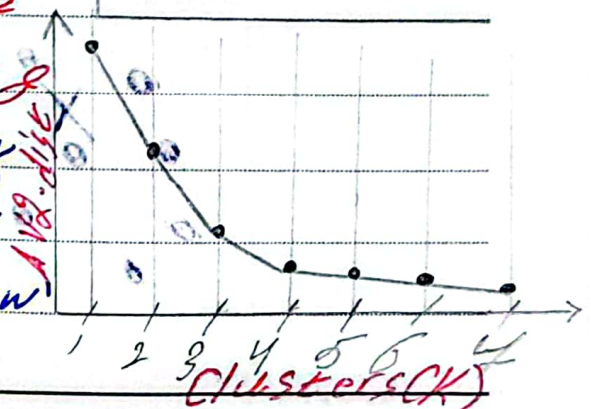


→ Large decreases at small K

→ Small decreases at large K

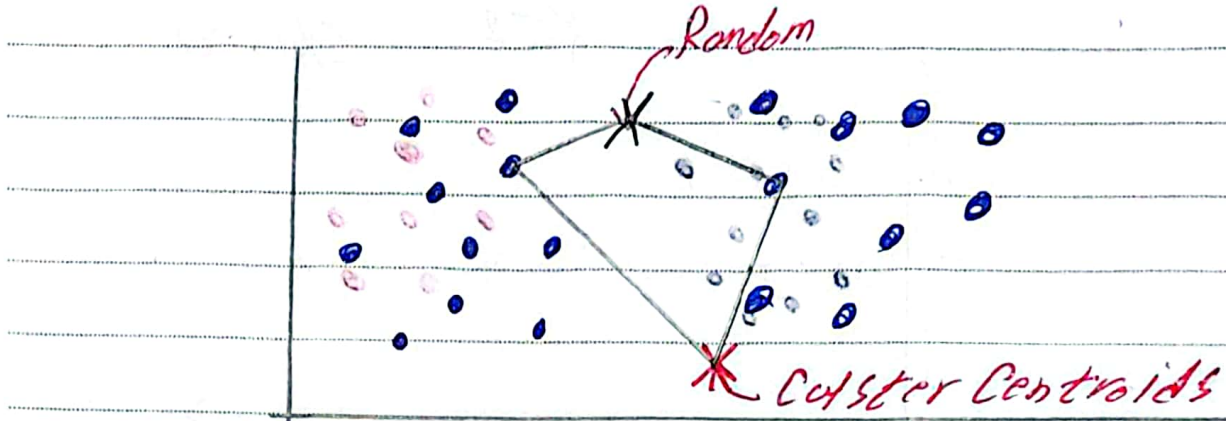
→ Best Choice around elbow

of Curve

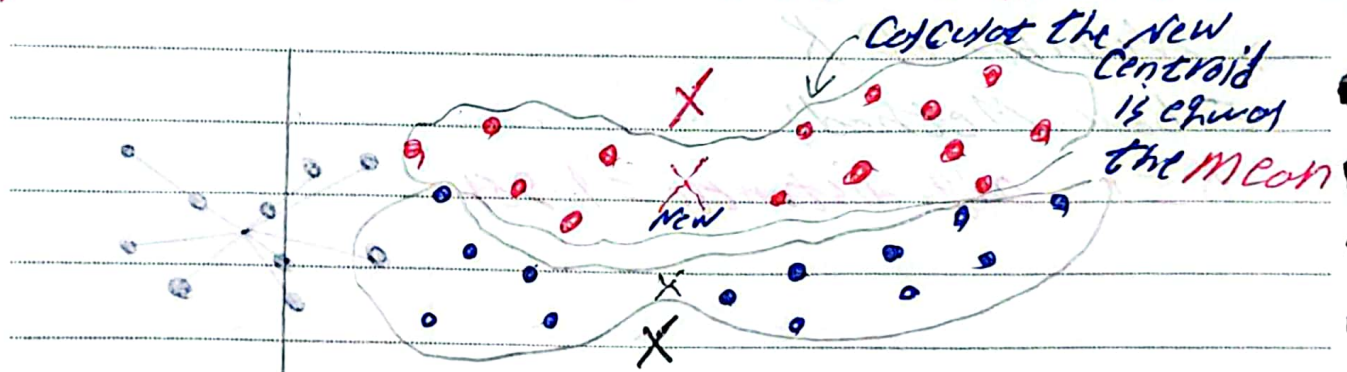


After select The number of K

② Randomly initialize Centroids:

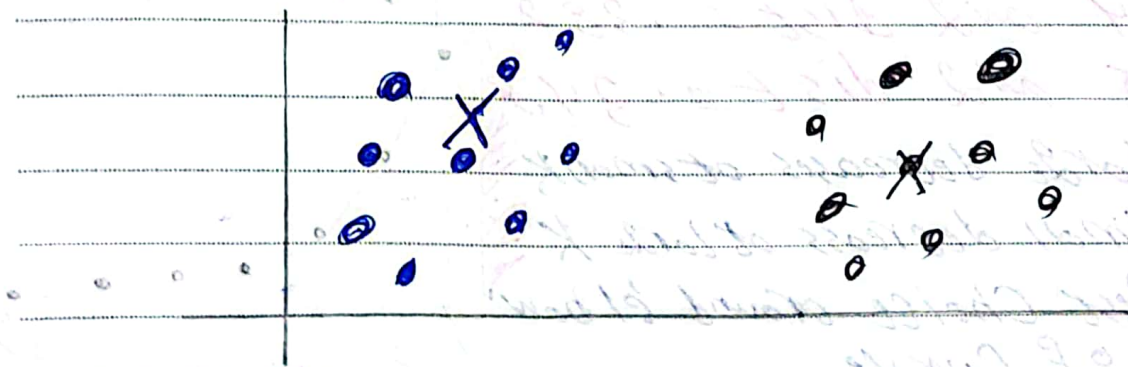


Assign each Point to its Closest Centroid



Then ReCompute the Centroids

→ And Repeated two Action to arrived to no Point Change his Color



* Feature scaling

→ Transform numerical features to similar scale

→ Types of Feature scaling

[1] Standardization (Z-score scaling)

Transform data so it has:

$$\text{mean} = 0$$

$$\text{Standard deviation} = 1$$

$$Z = \frac{X - \bar{M}}{\sigma}$$

used in many algorithms like:

- SVM, Logistic Regression

[2] Min-Max scaling (Normalization)

Transform values to a fixed range [0, 1]

$$X' = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

→ Min-max scaling (bad with outliers)

→ Robust scaling (best with outliers)

$$X' = \frac{X - \text{median}}{\text{IQR}}$$

~~Note to Avoid Data Leakage~~

~~Wrong Way scaling before Splitting~~

~~Scaler = standardScaler()~~

~~X_scaled = scaler.fit_transform(X)~~ # use on data

✓ Correct Way

X_train, X_test, y_train, y_test
= train_test_split(X, y)

Scaler = standardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)

~~Best Practice [Pipeline]~~

Pip = Pipeline

(("Scaler", standardScaler()),

("model", LogisticRegression())

)

Pip.fit(X_train, y_train)

التاريخ: / /

الموضوع:

* When use:

KNN, K-Means, SVM, Logistic
Neural Networks

→ Not Require scaling.

Random Forest, Decision Tree